

# Chapter 1

## Fundamental Concepts

### 1.1 Functional programming

Programming in a functional language consists of building definitions and using the computer to evaluate expressions. The primary role of the programmer is to construct a function to solve a given problem. This function, which may involve a number of subsidiary functions, is expressed in notation that obeys normal mathematical principles. The primary role of the computer is to act as an evaluator or calculator: its job is to evaluate expressions and print the results. In this respect, the computer acts much like an ordinary pocket calculator. What distinguishes a functional calculator from the humbler variety is the programmer's ability to make definitions to increase its powers of calculation. Expressions which contain occurrences of the names of functions defined by the programmer are evaluated by using the given definitions as simplification (or 'reduction') rules for converting expressions to printable form.

A characteristic feature of functional programming is that if an expression possesses a well-defined value, then the order in which a computer may carry out the evaluation does not affect the outcome. In other words, the meaning of an expression is its value and the task of the computer is simply to obtain it. It follows that expressions in a functional language can be constructed, manipulated and reasoned about, like any other kind of mathematical expression, using more or less familiar algebraic laws. The result, as we hope to justify, is a conceptual framework for programming which is at once very simple, very concise, very flexible and very powerful.

#### 1.1.1 Sessions and scripts

To illustrate the idea of using a computer as a calculator, imagine we are sitting at a terminal and the computer has indicated its willingness to evaluate an expression by displaying a prompt sign:

?

at the beginning of a blank line. We can then type an expression, followed by a newline character, and the computer will respond by printing the result of evaluating the expression, followed by a new prompt ? on a new line, indicating that the process can begin again with another expression.

One kind of expression we might type is a number:

? 42

42

Here, the computer's response is simply to redisplay the number we typed. The decimal numeral 42 is an expression in its simplest possible form and no further process of evaluation can be applied to it.

We might type a slightly more interesting kind of expression:

?  $6 \times 7$ 

42

Here, the computer can simplify the expression by performing the multiplication. In this book, we shall adopt common mathematical notations for writing expressions. In particular, the multiplication operator will be denoted by the sign  $\times$ . It may or may not be the case that a particular keyboard contains this sign, but we shall not concern ourselves in the text with how to represent mathematical symbols in a restricted character set.

We will not elaborate here on the possible forms of numerical and other kinds of expression that can be submitted for evaluation. They will be dealt with thoroughly in the following chapters. The important point to absorb for the moment is that one can just type expressions and have them evaluated. This sequence of interactions between user and computer is called a 'session'.

Now let us illustrate the second, and intellectually more challenging, aspect of functional programming: building definitions. A list of definitions will be called a 'script'. Here is a simple example of a script:

$$\begin{aligned} \text{square } x &= x \times x \\ \text{min } x \ y &= x, & \text{if } x \leq y \\ &= y, & \text{if } x > y \end{aligned}$$

In this script, two functions, named *square* and *min*, have been defined. The function *square* takes a value  $x$  as argument and returns the value of  $x$  multiplied by itself as its result. The function *min* takes two numbers,  $x$  and  $y$ , as arguments and returns the smaller value. For the present we will not discuss the exact syntax used for making definitions. Notice, however, that definitions are written as equations between certain kinds of expression; these expressions can contain variables, here denoted by the symbols  $x$  and  $y$ .

Having created a script, we can submit it to the computer and enter a session. For example, the following session is now possible:

```
? square (3 + 4)
```

```
49
```

```
? min 3 4
```

```
3
```

```
? square (min 3 4)
```

```
9
```

In effect, the purpose of a definition is to introduce a *binding* associating a given name with a given value. In the above script, the name *square* is associated with the function which squares its argument, and the name *min* is associated with the function which returns the smaller of its two arguments. A set of bindings is called an *environment* or *context*. Expressions are always evaluated within some context and can contain occurrences of the names found in that context. The evaluator will use the definitions associated with these names as rules for simplifying expressions.

Some expressions can be evaluated without the programmer having to provide a context. A number of operations may be given as primitive in that the rules of simplification are built into the evaluator. For example, we shall suppose the basic operations of arithmetic are provided as primitive. Other commonly useful operations may be provided in special libraries of predefined functions.

At any stage a programmer can return to the script in order to add or modify definitions. The new script can then be resubmitted to the computer to provide a new context and another session started.

For example, suppose we return to the script and add the definitions:

$$\begin{aligned} \textit{side} &= 12 \\ \textit{area} &= \textit{square side} \end{aligned}$$

These equations introduce two numerical constants, *side* and *area*. Notice that the definition of *area* depends on the previously defined function *square*. Having resubmitted the script, we can enter a session and type, for example:

```
? area
```

```
144
```

```
? min (area + 4) 150
```

```
148
```

To summarise the important points made so far:

1. Scripts are collections of definitions supplied by the programmer.
2. Definitions are expressed as equations between certain kinds of expression and describe mathematical functions.
3. During a session, expressions are submitted for evaluation; these expressions can contain references to the functions defined in the script.

## Exercises

**1.1.1** Using the function *square*, design a function *quad* which raises its argument to the fourth power.

**1.1.2** Define a function *max* which returns the greater of its two arguments.

**1.1.3** Define a function for computing the area of a circle with given radius *r* (use 22/7 as an approximation to  $\pi$ ).

## 1.2 Expressions and values

As we have seen, the notion of an expression is central in functional programming. There are many kinds of mathematical expression, not all of which are permitted in the notation we shall describe, but all possess certain characteristics in common. The most important feature of mathematical notation is that an expression is used solely to describe (or *denote*) a *value*. In other words, the meaning of an expression *is* its value and there are no other effects, hidden or otherwise, in any procedure for actually obtaining it. Furthermore, the value of an expression depends only on the the values of its constituent expressions (if any) and these subexpressions may be replaced freely by others possessing the same value. An expression may contain certain ‘names’ which stand for unknown quantities, but it is normal in mathematical notation to presume that different occurrences of the same name refer to the same unknown quantity (within obvious syntactic limits). Such names are usually called ‘variables’, but every mathematician understands that variables do *not* vary: they always denote the same quantity, provided we remain within the same context of the definitions associated with them. The characteristic property of mathematical expressions described here is called *referential transparency*.

Among the kinds of value an expression may denote are included: numbers, truth-values, characters, tuples, functions, and lists. All of these will be described in due course. As we shall see later on in the book, it is also possible to introduce new kinds of value and define operations for generating and manipulating them.

### 1.2.1 Reduction

The computer evaluates an expression by reducing it to its ‘simplest equivalent form’ and printing the result. The terms *evaluation*, *simplification*, and *reduction* will be used interchangeably to describe this process. We can give a brief flavour of the essence of reduction by considering the evaluation of the expression *square* (3 + 4). Suppose we let the sign  $\Rightarrow$  mean ‘reduces to’.

One possible reduction sequence is as follows:

$$\begin{aligned} \text{square } (3 + 4) &\Rightarrow \text{square } 7 \quad (+) \\ &\Rightarrow 7 \times 7 \quad (\text{square}) \\ &\Rightarrow 49 \quad (\times) \end{aligned}$$

In this sequence, the label  $(+)$  refers to a use of the built-in rule for addition,  $(\times)$  refers to a similar rule for multiplication, and  $(\text{square})$  refers to a use of the rule:

$$\text{square } x \Rightarrow x \times x$$

which is associated with the definition of *square* supplied by the programmer. The expression 49 cannot be further reduced, so that is the result printed by the computer.

The above sequence of reduction steps is not the only way to simplify the expression *square*  $(3 + 4)$ . Indeed, another sequence is as follows:

$$\begin{aligned} \text{square } (3 + 4) &\Rightarrow (3 + 4) \times (3 + 4) \quad (\text{square}) \\ &\Rightarrow 7 \times (3 + 4) \quad (+) \\ &\Rightarrow 7 \times 7 \quad (+) \\ &\Rightarrow 49 \quad (\times) \end{aligned}$$

In this reduction sequence the rule for *square* is applied first, but the final result is the same. A fuller account of reduction, including a discussion of different reduction strategies, will be given in Chapter 6. The point to grasp here is that expressions can be evaluated by a basically simple process of substitution and simplification, using both primitive rules and rules supplied by the programmer in the form of definitions.

It is important to be clear about the distinction between values and their representations by expressions. The simplest equivalent form of an expression, whatever that may be, is *not* a value but a representation of it. Somewhere, in outer space perhaps, one can imagine a universe of abstract values, but on earth they can only be recognised and manipulated by their representations. There are many representations for one and the same value. For example, the abstract number forty-nine can be represented by the decimal numeral 49, the roman numeral XLIX, the expression  $7 \times 7$ , as well as infinitely many others. Computers usually operate with the binary representation of numbers in which forty-nine may be represented by the pattern 0000000000110001 of 16 bits.

We shall say an expression is *canonical* (or in *normal form*) if it cannot be further reduced. A value is printed as its canonical representation. Notice that the notion of a canonical expression is dependent both on the syntax given for forming expressions and the precise definition of the permissible reduction rules. Some values have no canonical representations, others have no finite ones. For example, the number  $\pi$  has no finite decimal representation.

It is possible to get a computer to print out the decimal expansion of  $\pi$  digit by digit, but the process will never terminate.

Some expressions cannot be reduced at all. In other words, they do not denote well-defined values in the normal mathematical sense. For instance, supposing the operator  $/$  denotes numerical division, the expression  $1/0$  does not denote a well-defined number. A request to evaluate  $1/0$  may cause the evaluator to respond with an error message, such as ‘attempt to divide by zero’, or go into an infinitely long sequence of calculations without producing any result. In order that we can say that, without exception, every (well-formed) expression denotes a value, it is convenient to introduce a special symbol  $\perp$ , pronounced ‘bottom’, to stand for the undefined value. In particular, the value of  $1/0$  is  $\perp$  and we can assert  $1/0 = \perp$ . The computer is not expected to be able to produce the value  $\perp$ . Confronted with an expression whose value is  $\perp$ , the computer may give an error message, or it may remain perpetually silent. Thus,  $\perp$  is a special kind of value, rather like the special value  $\infty$  in mathematical calculus. Like special values in other branches of mathematics,  $\perp$  can be admitted to the universe of values only if we state precisely the properties it is required to have and its relationship with other values. We shall not go into the properties of  $\perp$  for a while, but for now merely note the reasons for its existence and its special status.

## Exercises

**1.2.1** Count the number of different ways that:

$$\text{square}(\text{square}(3 + 7))$$

can be reduced to normal form.

**1.2.2** Consider the definition:

$$\text{three } x = 3$$

In how many ways can  $\text{three}(3 + 4)$  be reduced to normal form?

**1.2.3** Imagine a language of expressions for representing integers defined by the syntax rules: (i) *zero* is an expression; (ii) if  $e$  is an expression, then so are  $(\text{succ } e)$  and  $(\text{pred } e)$ . An evaluator reduces expressions in this language by applying the following rules repeatedly until no longer possible:

$$\begin{array}{ll} (\text{succ } (\text{pred } e)) & \Rightarrow e \quad (\text{succ.1}) \\ (\text{pred } (\text{succ } e)) & \Rightarrow e \quad (\text{pred.1}) \end{array}$$

Simplify the expression

$$(\text{succ}(\text{pred}(\text{succ}(\text{pred}(\text{pred } \text{zero}))))))$$

In how many ways can the reduction rules be applied to this expression? Do they all lead to the same final result? Prove that the process of reduction must terminate for all given expressions. (*Hint*: Define an appropriate notion of expression size, and show that reduction does indeed reduce size.)

Suppose an extra syntactic rule is added to the language: (iii) if  $e_1$  and  $e_2$  are expressions, then so is  $(add\ e_1\ e_2)$ . The corresponding reduction rules are:

$$\begin{aligned} (add\ zero\ e_2) &\Rightarrow e_2 && (add.1) \\ (add\ (succ\ e_1)\ e_2) &\Rightarrow (succ\ (add\ e_1\ e_2)) && (add.2) \\ (add\ (pred\ e_1)\ e_2) &\Rightarrow (pred\ (add\ e_1\ e_2)) && (add.3) \end{aligned}$$

Simplify the expression:

$$(add\ (succ\ (pred\ zero))\ zero).$$

Count the number of different ways the reduction rules can be applied to the above expression. Do they always lead to the same final result? Prove that the process of reduction must always terminate for any given initial expression. (*Hint*: Extend the notion of expression size.)

**1.2.4** Imagine a language of finite sequences of 0 and 1. The rules for simplifying strings in this language are given by:

$$\begin{aligned} 1??x &\Rightarrow x1101 \\ 0??x &\Rightarrow x00 \end{aligned}$$

In these rules, the variable  $x$  denotes an arbitrary sequence of 0s and 1s and the sign ‘?’ denotes a single 0 or 1. Reduce the following expressions to canonical form:

$$1110 \quad 10 \quad 1110100$$

Construct an expression for which the reduction process does not terminate. (Such a system of reduction rules is known as a *Post Normal System*; see Minsky [1] for further details. Although it is easy to construct strings that ‘loop’, it is an open problem whether or not there is an initial string on which the above system fails to terminate by producing an infinite number of successively larger strings.)

## 1.3 Types

In the notation we are going to describe, the universe of values is partitioned into organised collections, called *types*. Types can be divided into two kinds. Firstly, there are *basic* types whose values are given as primitive. For example, numbers constitute a basic type (the type *num*), as do truth-values (the type *bool*) and characters (the type *char*). Secondly, there are *compound* (or *derived*) types, whose values are constructed from those of other

types. Examples of derived types include:  $(num, char)$ , the type of pairs of values, the first component of which is a number and the second a character;  $(num \rightarrow num)$ , the type of functions from numbers to numbers; and  $[char]$ , the type of lists of characters. Each type has associated with it certain operations which are not meaningful for other types. For instance, one cannot sensibly add a number to a character or multiply two functions together.

It is an important principle of the notation we are going to describe that every well-formed expression can be assigned a type that can be deduced from the constituents of the expression alone. In other words, just as the value of an expression depends only on the values of its component expressions, so does its type. This principle is called *strong-typing*.

The major consequence of the discipline imposed by strong-typing is that any expression which cannot be assigned a 'sensible' type is regarded as not being well-formed and is rejected by the computer before evaluation. Such expressions have *no* value: they are simply regarded as illegal.

Here is an example of a script which contains a definition that cannot be assigned a sensible type:

$$\begin{aligned} ay\ x &= 'A' \\ bee\ x &= x + ay\ x \end{aligned}$$

The expression 'A' in this script denotes the character A. For any  $x$ , the value of  $ay\ x$  is 'A' and so has type *char*. Since  $+$  is reserved to denote the operation of numerical addition, the right-hand side of the definition of *bee* is not well-typed: one cannot add characters numerically. It follows that the function *bee* does not possess a sensible type, and the script is rejected by the computer. (On the other hand, the function *ay* does possess a sensible type; we shall see what it is in the next section.)

There are two stages of analysis when an expression is submitted for evaluation. The expression is first checked to see whether it conforms to the correct syntax laid down for expressions. If it does not, the computer signals a syntax error. This stage is called *syntax-analysis*. If it does, then the expression is analysed to see if it possesses a sensible type. This stage is called *type-analysis*. If the expression fails to pass this stage, the computer signals a type error. Only if the expression passes both stages can the process of evaluation begin. Similar remarks apply to definitions before a script is accepted.

Strong typing is important because adherence to the discipline can help in the design of clear and well-structured programs. What is more, a wide range of logical errors can be trapped by any computer which enforces it.

## 1.4 Functions and definitions

The most important kind of value in functional programming is a function value. Mathematically speaking, a function  $f$  is a rule of correspondence



which associates with each element of a given type  $A$  a unique member of a second type  $B$ . The type  $A$  is called the *source* type, and  $B$  the *target* type of the function. We will express this information by writing:

$$f :: A \rightarrow B$$

This formula asserts that the type of  $f$  is  $A \rightarrow B$ . In other words, the type-expression  $A \rightarrow B$  denotes a type whenever  $A$  and  $B$  do, and describes the type of functions from  $A$  to  $B$ .

A function  $f :: A \rightarrow B$  is said to take *arguments* in  $A$  and return *results* in  $B$ . If  $x$  denotes an element of  $A$ , then we write  $f(x)$ , or just  $fx$ , to denote the result of *applying* the function  $f$  to  $x$ . This value is the unique element of  $B$  associated with  $x$  by the rule of correspondence for  $f$ . The former notation,  $f(x)$ , is the one normally employed in mathematics to denote functional application, but the brackets are not really necessary and we shall use the second form,  $fx$ , instead. However, when formal expressions are mixed in with running prose we shall often surround them with brackets to aid the eye. For example, we write  $(fx)$  rather than  $fx$ .

We shall be careful never to confuse a function with its application to an argument. In some mathematics texts one often finds the phrase ‘the function  $f(x)$ ’, when what is really meant is ‘the function  $f$ ’. In such texts, functions are rarely considered as values which may themselves be used as arguments to other functions and the usage causes no confusion. In functional programming, however, functions are values with exactly the same status as all other values; in particular, they can be passed as arguments to functions and returned as results. Accordingly, we cannot afford to be casual about the difference between a function and the result of applying it to an argument.

It is important to keep in mind the distinction between a function value and a particular definition of it. There are many possible definitions for one and the same function. For instance, we can define the function which doubles its argument in the following two ways:

$$\begin{aligned} \text{double } x &= x + x \\ \text{double}' x &= 2 \times x \end{aligned}$$

The two definitions describe different procedures for obtaining the correspondence, but *double* and *double'* denote the same function and we can assert  $\text{double} = \text{double}'$  as a mathematical truth. Regarded as procedures for evaluation, one definition may be more or less ‘efficient’ than the other, in the sense that the evaluator may be able to reduce expressions of the form  $(\text{double } x)$  more or less quickly than expressions of the form  $(\text{double}' x)$ . However, the notion of efficiency is not one which can be attached to function values themselves. Indeed, it depends on the given form of the definition and the precise characteristics of the mechanism that evaluates it.

### 1.4.1 Type information

When defining functions in a script, it is allowable to include type information about the function. For example, we can define the function *square* in a script by writing:

$$\begin{aligned} \textit{square} &:: \textit{num} \rightarrow \textit{num} \\ \textit{square } x &= x \times x \end{aligned}$$

Although it is often good practice to include them, function definitions do not have to be accompanied by type declarations. The type of the function can be inferred from its defining equation alone. This is an instance of the strong-typing discipline mentioned in the previous section. The operator  $\times$  is reserved exclusively for the multiplication of numeric quantities, so the type assignment  $\textit{square} :: \textit{num} \rightarrow \textit{num}$  can be deduced mechanically.

Some functions have very general source and target types. Consider the following definition:

$$\textit{id } x = x$$

This equation defines the identity function: it maps every member of the source type to itself. Its type is therefore  $A \rightarrow A$  for some suitable type  $A$ . But every type  $A$  is suitable, since no particular property of the elements of  $A$  is required in the definition of *id*. The problem of giving a sensible type to *id* is solved by introducing *type variables*. The type assigned to *id* is  $\alpha \rightarrow \alpha$ . Here  $\alpha$  denotes a type variable. We shall use greek letters  $\alpha, \beta, \gamma, \dots$ , and so on, to denote type variables. Like other kinds of variable, a type variable can be instantiated to different types in different circumstances. For instance, the expression  $(\textit{id } 3)$  is well-formed and has type *num* because *num* can be substituted for  $\alpha$  in the type of *id*, yielding a  $(\textit{num} \rightarrow \textit{num})$  version. Similarly, the expression  $(\textit{id } \textit{square})$  is well-formed and has type  $(\textit{num} \rightarrow \textit{num})$  because  $(\textit{num} \rightarrow \textit{num})$  (the type of the function *square*) can be substituted for  $\alpha$ . Finally, the expression  $(\textit{id } \textit{id})$  is also well-formed because the type  $(\alpha \rightarrow \alpha)$  can itself be substituted for  $\alpha$ . The type of  $(\textit{id } \textit{id})$  is therefore  $(\alpha \rightarrow \alpha)$ . And, of course, we have  $\textit{id } \textit{id} = \textit{id}$ .

Here is another example of a valid definition whose associated type contains variables. Recall the definition of the function *ay* from the previous section:

$$\textit{ay } x = 'A'$$

The type associated with *ay* is  $\textit{ay} :: \alpha \rightarrow \textit{char}$ . The source type of *ay* can be any type at all.

We now have the beginnings of a language of expressions that denote types. This language contains constant expressions, such as *num* or *char*, variables, such as  $\alpha$  and  $\beta$ , and operators, such as  $\rightarrow$ . If such an expression does contain variables, then we say that it denotes a *polymorphic* type. In particular, the functions *id* and *ay* have polymorphic types.

### 1.4.2 Forms of definition

In many situations we may want to define the value of a function by case analysis. Consider again the function *min* from Section 1.1:

$$\begin{aligned} \min x y &= x, \text{ if } x \leq y \\ &= y, \text{ if } x > y \end{aligned}$$

This definition consists of two expressions, each of which is distinguished by boolean-valued expressions, called *guards*. A boolean-valued expression is an expression which evaluates to one of the truth-values *True* or *False*. The first alternative of the definition says that the value of  $(\min x y)$  is defined to be  $x$ , provided the expression  $x \leq y$  evaluates to *True*. The second alternative says that  $(\min x y)$  is defined to be  $y$  provided the expression  $x > y$  evaluates to *True*. The two cases,  $x \leq y$  and  $x > y$ , exhaust all possibilities, so the value of *min* is defined for all numbers  $x$  and  $y$ . It does not matter in which order we write the alternatives because the two cases are disjoint.

Another way to define *min* is to write:

$$\begin{aligned} \min x y &= x, \text{ if } x \leq y \\ &= y, \text{ otherwise} \end{aligned}$$

The special word ‘otherwise’ can be thought of as a convenient abbreviation for the condition which returns the value *True* when all previous guards return the value *False*.

The final piece of notation we shall introduce here is called a *local* definition. In mathematical descriptions one often finds an expression qualified by a phrase of the form ‘where ...’. For instance, one might find ‘ $f(x, y) = (a + 1)(a + 2)$ , where  $a = (x + y)/2$ ’. The same device can be used in a formal definition:

$$\begin{aligned} f(x, y) &= (a + 1) \times (a + 2) \\ &\text{where } a = (x + y)/2 \end{aligned}$$

Here, the special word ‘where’ is used to introduce a local definition whose context (or *scope*) is the expression on the right-hand side of the definition of  $f$ . Notice that the whole of the where-clause is indented to show it is part of this expression.

A local definition can be used in conjunction with a definition by case analysis. Consider the following:

$$\begin{aligned} f x y &= x + a, \text{ if } x > 10 \\ &= x - a, \text{ otherwise} \\ &\text{where } a = \text{square } (y + 1) \end{aligned}$$

In this definition, the where-clause qualifies *both* parts of the right-hand side. Although, for readability, there are two occurrences of the  $=$  sign in the case analysis for  $f$ , there is only one equation and the scope of the where-clause includes all of it.

### 1.4.3 Currying

Consider again the definition of *min*:

$$\begin{aligned} \text{min } x \ y &= x, \text{ if } x \leq y \\ &= y, \text{ if } x > y \end{aligned}$$

Notice that the arguments to *min* are written without brackets and an intervening comma. We can, if we like, add brackets and write:

$$\begin{aligned} \text{min}'(x, y) &= x, \text{ if } x \leq y \\ &= y, \text{ otherwise} \end{aligned}$$

The two functions, *min* and *min'*, are very closely related, but there is a subtle difference: they have different types. The function *min'* takes a single argument which is a structured value consisting of a *pair* of numbers; its type is given by:

$$\text{min}' :: (\text{num}, \text{num}) \rightarrow \text{num}$$

The function *min*, on the other hand, takes two arguments one at a time. Its type is given by:

$$\text{min} :: \text{num} \rightarrow (\text{num} \rightarrow \text{num})$$

In other words, *min* is a function which takes a number and returns a function (from numbers to numbers). For each value of *x* the expression (*min x*) denotes a function which takes an argument *y* and returns the minimum of *x* and *y*.

Here is another example. The function *add*, defined by:

$$\text{add } x \ y = x + y$$

also has type  $\text{num} \rightarrow (\text{num} \rightarrow \text{num})$ . For each *x*, the function (*add x*) ‘adds *x* to things’. In particular, (*add 1*) is the successor function which increments its argument by 1, and (*add 0*) is the identity function on numbers.

This simple device for replacing structured arguments by a sequence of simple ones is known as ‘currying’, after the American logician H. B. Curry. One advantage of currying is that it reduces the number of brackets which have to be written in expressions (an aspect of the notation the reader will quickly grow to appreciate). For currying to work properly in a consistent manner, we require that the operation of functional application associates to the left. That is, *min x y* means (*min x*) *y* and not *min*(*x y*). As an operator, functional application has a very ‘quiet’ notation, being represented by just a space. In formal expressions this quietness improves readability, and with currying we can exploit quietness to the full.

We have now said enough about functions, types and definitions to enable simple scripts to be written. Further material on functions will be found at the end of the next chapter.

## Exercises

**1.4.1** Describe one appropriate type for the definite integral function of mathematical analysis, as used in the phrase ‘the integral of  $f$  from  $a$  to  $b$ ’.

**1.4.2** Give examples of functions with the following types:

$$\begin{aligned} & (num \rightarrow num) \rightarrow num \\ & num \rightarrow (num \rightarrow num) \\ & (num \rightarrow num) \rightarrow (num \rightarrow num) \end{aligned}$$

**1.4.3** Give a definition of a function  $sign :: num \rightarrow num$  which returns 1 if its argument is positive,  $-1$  if its argument is negative, and 0 otherwise.

**1.4.4** Suggest possible types for the following functions:

$$\begin{aligned} one\ x &= 1 \\ apply\ f\ x &= f\ x \\ compose\ f\ g\ x &= f\ (g\ x) \end{aligned}$$

## 1.5 Specifications and implementations

In computer programming, a *specification* is a mathematical description of the task a program is to perform, while an *implementation* is a program that satisfies the specification. Specifications and implementations are quite different in nature and serve different purposes. Specifications are expressions of the programmer’s intent (or client’s expectations) and their purpose is to be as brief and clear as possible; implementations are expressions for execution by computer and their purpose is to be efficient enough to execute within the time and space available. The link between the two is the requirement that the implementation satisfies, or meets, the specification, and the serious programmer is obliged to provide a *proof* that this is indeed the case.

A specification for a function value is some statement of the intended relationship between argument values and results. A simple example is given by the following specification of a function  $increase :: num \rightarrow num$ :

$$increase\ x > square\ x$$

for all  $x \geq 0$ . This just says that the result of *increase* should be greater than the square of its argument, whenever the argument is greater than or equal to zero.

Here is one possible implementation of *increase*:

$$increase\ x = square\ (x + 1)$$

This is a valid definition in our programming notation. The proof that this definition of *increase* satisfies the specification is as follows: assuming  $x \geq 0$ , we have:

$$\begin{aligned}
 \text{increase } x &= \text{square } (x + 1) && (\text{increase}) \\
 &= (x + 1) \times (x + 1) && (\text{square}) \\
 &= x \times x + 2 \times x + 1 && (\text{algebra}) \\
 &> x \times x && (\text{assumption: } x \geq 0) \\
 &= \text{square } x && (\text{square})
 \end{aligned}$$

Here we have invented a definition of *increase* first, and afterwards verified that it meets the specification. Clearly, there are many other functions which will satisfy the specification and, since this is the only requirement, all are equally good.

One way of specifying a function is to state the rule of correspondence explicitly. The functional notation we shall describe can be very expressive, and it is often possible to write down a formal definition within the notation which will actually serve as the specification. This specification can then be executed directly. However, it may prove so grossly inefficient that the possibility of execution will be of theoretical interest only. Having written an executable specification, the programmer is not necessarily relieved of the burden (or pleasure) of producing an equivalent but acceptably efficient alternative.

The problem of showing that formal definitions meet their specifications can be tackled in a number of ways. One approach, illustrated above, is to design the definition first and afterwards verify that the necessary conditions are satisfied. Another approach, which can lead to clearer and simpler programs, is to systematically develop (or *synthesise*) the definition from the specification. For example, if we look at the specification of *increase* we may argue that since  $x + 1 > x$  for all  $x$ , we have:

$$\text{square } x + 1 > \text{square } x$$

and so we can define:

$$\text{increase } x = \text{square } x + 1$$

In fact, the new definition of *increase* satisfies a stronger specification than required, since  $\text{increase } x > \text{square } x$  for all possible values of  $x$ , including negative ones. This does not invalidate the definition because any implementation that gives more than is required is at least as good as one that does not. We shall see other, more convincing, examples of systematic program development as we progress.

This paradigm of software development – first write a clear specification, then develop an acceptably efficient implementation – is the focus of active research in computation and should not be taken as a cut-and-dried method,

applicable in all circumstances. Two potential sources of difficulty are that the formal specification may not match our informal intentions, and the proof that the implementation matches the specification may be so large or complicated that it cannot be guaranteed to be free of error. Nevertheless, by trying to follow the approach whenever we can, the reliability of programs can be greatly increased.

## Exercises

**1.5.1** Using any suitable notation, write down a specification of a function *isSquare* that determines whether or not its argument is an integer which is the square of another integer. Suppose the value of (*intsqrt* *x*) is the largest integer which is no greater than  $\sqrt{x}$ . Will the following definition meet your specification?

$$\textit{isSquare } x = (\textit{square } (\textit{intsqrt } x) = x)$$

**1.5.2** Write down a precise specification of the function *intsqrt* mentioned in the previous question.